Changes since the end of iteration 3

- ros2 control config:
    - Angular Z max velocity and max acceleration raised to cover MPPI's limits.
    - Velocity rolling window size is 3.
    - Wheel separation set properly and verified with a tape measure.
- urdf:
    - Wheel radius corrected to match ros2 control config. Verified with a tape measure; ros2 control config had the right value.
- Hardware interface and firmware:
    - Board B now sends wheel angvels to board A inside its PID loop. I figured this is fine because esp_now_send does not block and is pinned to one core. In board A's receive callback, it simply copies the received angvels to a global atomic.
    - Switched board B for a different ESP32 dev board. This fixed all the "congestion" I was talking about in the comments.
    - Board A never prints on its own. In the hw interface read(), it first writes a single "g" character and a newline to signal to board A that it must print the latest angvels. This ensures that the B->A->ROS latency path is limited only by rtlatency and ros2 control; there are no "reading the previous cycle's data" shenanigans.
    - the actual "rtlatency" in the code is now meaningless, since it relied on the ping-pong communication from before, which is now gone. It should always be zero now, and will be removed in the future. Do not look at it.

PID tuning

Some attempts were made to optimize the PID controller, but no better gain configuration was found than the one currently (p=2.3, i=6.2, d=0) at a 5 Hz update rate. The update rate was not changed, only the P and I gains.

Raising P marginally improves the high-magnitude (~1.0 rad/s) step responses, but makes oscillations happen at lower step magnitudes. Any amount of D gain simply introduces too much noise and leads to unpredictable behavior; if D gain is necessary, then we must lowpass-filter the encoder signals or employ other techniques.

Test Performed

The following topics were bagged:
ros2 bag record -o latency_bag2 /diffbot_base_controller/cmd_vel /diffbot_base_controller/odom /tf /dynamic_joint_states /tf_static /rosout /gpio_controller/gpio_states /imu/data /odometry/filtered/local

The robot was given ten step responses of various amplitudes using rqt.

In your report…

- Informational latency (time of first motor movement minus time of step change in cmd vel)
- Full latency (time of reaching 95% of setpoint minus time of step change in cmd vel)
- Similar graphs to the previous latency report.